PRESIDENCE DE LA REPUBLIQUE

MINISTERE DE LA PLANNIFICATION DU DEVELOPPEMENT ET DE LA COOPERATION



Institut Africain d'Informatique

Etablissement Inter-Etats d'Enseignement Supérieur
Représentation du Togo (IAI-TOGO)
07 **BP** 12456 Lomé 07 **Tél.** : (+228) 22.20.47.00
E-mail : iaitogo@iai-togo.com **Site Web**: www.iai-togo.tg

Support de cours

INITIATION A L'ALGORITHMIQUE

Année académique 2022-2023

AVANT-PROPOS

L'algorithme est l'étude et la définition des règles techniques qui sont impliquées dans la conception d'algorithmes. Ce sont des processus systématiques de résolution de problèmes permettant de décrire précisément des étapes pour résoudre un problème algorithmique. En résumé un algorithme est une suite finie et non-ambiguë d'instructions permettant de donner la réponse à un problème.

Ce support est destiné aux étudiants de première année de Licence Informatique ou d'Ingénierie des Travaux Informatiques. Il demeure en tout état de cause un support ; de ce fait, invite les étudiants à mettre leurs talents et leurs sérieux à un travail soutenu et à la recherche pour ainsi parfaire leur connaissance et renforcer leur capacité dans la maitrise de la logique.

Il est structuré en quatre (04) chapitres :

- Structures de données fondamentales ;
- Composantes élémentaires des algorithmes ;
- Principes algorithmiques de tri des tableaux ;
- Notions sur les algorithmes récursifs.

Un cahier d'exercice complète ce support pour des travaux dirigés et pratiques afin de rendre les étudiants plus opérationnels en algorithme.

PROGRAMME

AVANT-PROPOS

PROGRAMME

Chapitre 1 : STRUCTURES DE DONNEES FONDAMENTALES

I. NOTION DE BASE

II. LES VARIABLES ALGORITHMIQUES

III. CONVENTIONS SYNTAXIQUES

Chapitre 2 : COMPOSANTES ELEMENTAIRES DES ALGORITHMES

I. LES INSTRUCTIONS DE BASE

II. LES STRUCTURES CONDITIONNELLES

Chapitre 3 : LES TABLEAUX ET LES FICHIERS

I. LA GESTION DES TABLEAUX

II. LES TABLEAUX À UNE DIMENSION

III. LES TABLEAUX À DEUX DIMENSION

IV. OPÉRATIONS DE TRI SUR LES TABLEAUX

V. NOTION DE FICHIER

Chapitre 4 : LES PROCÉDURES ET LES FONCTIONS

I. LES PROCÉDURES

II. LA NOTION DE FONCTION

III. LA PROGRAMMATION PAR MODULES

Chapitre 1 : STRUCTURES DE DONNEES FONDAMENTALES

I. NOTION DE BASE

INTRODUCTION

Programmer consiste à acquérir une démarche d'esprit et une méthodologie. C'est définir précisément le problème à résoudre, définir peu à peu une solution et après seulement l'exprimer dans un langage de programmation.

L'écriture d'un programme informatique est une tâche exigeante par sa rigueur. Les ordinateurs exigent perfection et précision dans les instructions qui leur sont données car ils ne font que ce que l'on leur demande de faire.

L'algorithme doit donc être perçu comme un outil favorisant la réflexion, l'analyse d'un problème sans se soucier des difficultés inhérentes au langage de programmation.

OBJECTIFS

L'algorithme s'appuie sur deux concepts fondamentaux :

- La calculabilité : c'est la réalisation d'un calcul sur des nombres et des caractères ;
- La notion de machine abstraite : c'est une théorie sur les calculs réalisés sur tous les ordinateurs.

La notion d'algorithme se comprend comme l'écriture d'un procédé de calcul effectué sur une machine abstraite. La notion de programme est perçue comme la transcription d'un algorithme dans un langage de programmation.

1. DÉFINITIONS

1.1 INFORMATIQUE

L'informatique se définit selon trois (03) aspects :

- **De façon très concrète,** c'est la technologie des ordinateurs, domaine qui recouvre tous les composants utilisables par l'ordinateur éventuellement placés dans un réseau ;
- **D'une façon plus large,** c'est le marché des produits logiciels ou matériels, qu'ils s'agissent de systèmes locaux ou de réseaux.
- **Ou encore,** c'est la science des modules calculables, la science de ce qui se calcule sur et avec les ordinateurs, l'objet de l'algorithmique ;
- **Enfin,** c'est la technologie de l'information et de la communication, c'est le numérique, c'est le digital.

1.2 ALGORITHMIQUE

L'algorithmique est la science des algorithmes utilisés notamment en informatique (Petit Larousse). Elle consiste concrètement en la conception et la rédaction de l'algorithme.

1.3 ALGORITHME

Le mot vient du nom du mathématicien arabe du 9e siècle MUHAMED IBN ALKHAREZIN. Il en existe beaucoup de définitions. A titre d'exemple en voici quelques-unes :

- C'est une succession de calculs numériques (Larousse) ;
- C'est une suite finie d'opérations élémentaires constituant un schéma de calcul ou de résolution d'un problème (Petit Larousse) ;
- C'est un ensemble de règles opératoires propres à un calcul (Petit Robert) ;
- C'est un procédé spécial de résolution d'un certain type de problèmes (dictionnaire des mathématiques).

Cinq (05) caractéristiques de l'algorithme sont les suivantes :

- Il est fini et se termine après un nombre fini d'opérations ;
- Il est défini sans ambigüité ;
- Il manipule des objets qui doivent être définis de façon précises ;
- Il doit avoir au moins un résultat ;
- Il doit être effectif : toutes les opérations doivent pouvoir être effectuées exactement et dans un temps fini par homme utilisant des moyens manuels.

Les actions ou opérations d'un algorithme sont décrites à l'aide de mots respectant des règles syntaxiques et de présentations qui en facilite la compréhension. Toutes ces règles définissent la notation algorithmique.

En résumé, le schéma suivant résume la place de l'algorithme dans la programmation des ordinateurs.

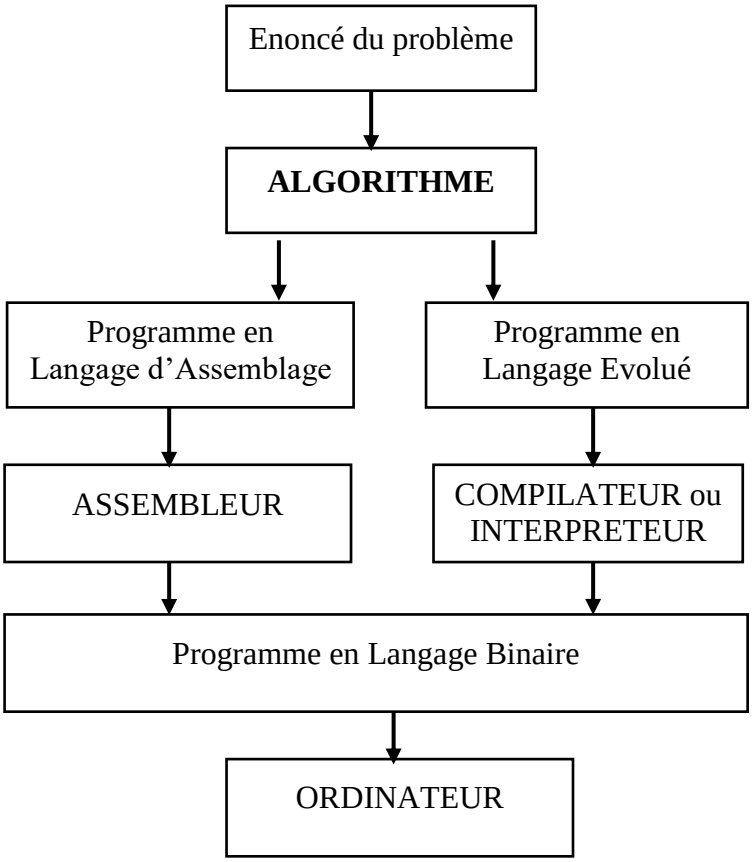


Figure 1: Communication Homme-Machine

L'algorithme est l'étude des processus de traitement. Les algorithmes permettent de représenter des schémas de programmation pour résoudre des problèmes complexes dans les domaines administratifs, comptables, scientifiques, industriels, etc.

Pour indiquer à une personne le chemin à suivre pour parvenir à l'endroit demandé, on remarque :

- Qu'il est nécessaire de connaître le lieu même où l'on se trouve (on ne peut fournir le renseignement si l'on est soi-même perdu) : c'est la connaissance de **l'état initiale** ;
- Que le demandeur doit être capable de réaliser un certain nombre d'actions (reconnaitre la droite et la gauche, lire le nom des rues …) ;
- Qu'il est nécessaire de connaitre l'endroit auquel la personne désir se rendre : c'est la connaissance de **l'état final.**

Nous pouvons enfin définir l'algorithme comme étant :

> Un ensemble de règles opératoires rigoureuses, ordonnant à un processus particulier d'exécuter dans un ordre déterminé un nombre fini d'opérations élémentaires, pour résoudre tous les problèmes d'un type donné.

Les actions d'un algorithme s'articulent à l'aide de structure de base. Ces structures de base sont :

- La structure séquentielle ou linéaire : ensemble d'opérations à la suite les unes des autres ;
- La structure conditionnelle : ensemble d'opérations alternatives ou répétitives soumises à une condition.

1.4 PROGRAMMATION

La programmation consiste à coder (traduire) dans un langage de programmation un algorithme en respectant scrupuleusement la syntaxe du langage choisi. Le programme ainsi obtenu est constitué d'un ensemble d'instructions capables d'être exécuté par le processeur d'un ordinateur.

1.5 INSTRUCTION

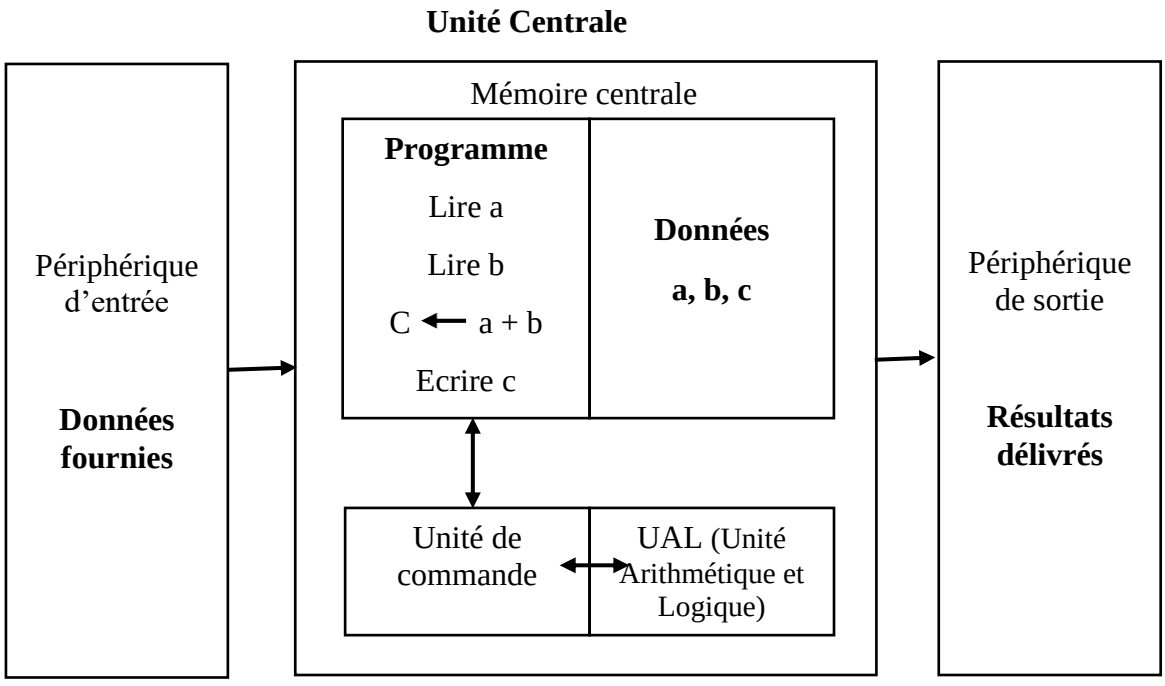
L'instruction, nommée aussi primitive, est un mot reconnu et interprété par un ordinateur capable d'effectuer une action élémentaire (lire, écrire, affecter, calculer …).

1.6 ORDINATEUR

EN 1945 le mathématicien hongrois JOHN VON NEUMANN écrit les cinq composants clés de ce qui allait devenir l'architecture de l'ordinateur moderne : « **l'architecture de NEUMANN** ».

Pour être efficace et polyvalent, écrit-il un calculateur doit posséder une unité arithmétique et logique, une unité centrale de commande pour orchestrer les traitements, une mémoire, une unité d'entré et une unité de sortie. Il faut également remarquer qu'un tel système doit avoir recours à la numération binaire, fonctionner électroniquement et accomplir les tâches une à une.

1.7 SCHEMA D'EXECUTION D'UN PROGRAMME PAR UN ORDINATEUR



1.8 LANGAGE DE PROGRAMMATION

C'est un langage compréhensif par l'ordinateur. L'ordinateur ne comprend que le langage binaire. Il existe un programme de traduction, compilateur ou traducteur (interpréteur), qui code en langage machine (code binaire) ce que le programmeur a écrit.

2. DECOUVERTE DE QUELQUES OUTILS DE PROGRAMMATION

2.1. Exposés sur quelques outils de programmation

(Quatre groupes de 3 étudiants seront constitués)

- Pascal
- C
- Python

2.2 Tableau comparatif

(Restitution après recherche par les étudiants)

3. LES PHASES DE PROGRAMMATION

La rédaction d'un programme suit une démarche progressive que l'on résume aux phases suivantes

- **Phase de spécification** : c'est la description exacte du problème à résoudre ;
- **Phase de conception** : c'est l'analyse structurée du problème et la recherche d'une solution ;
- **Phase de codage** : c'est l'écriture du programme dans un langage de programmation ;
- **Phase de correction** : c'est le test et la mise au point du programme ;
- **Phase de maintenance** : c'est la mise à jour du programme.

3.1 PHASE DE SPECIFICATION

La définition exacte des taches à résoudre est en général une étape tellement évidente que bien des programmeurs la négligent. Cela a pour effet de prendre en compte des hypothèses implicites qui sont celles des informaticiens et non celles des utilisateurs. Pour éviter ces hypothèses implicites, il est important de préciser les spécifications suivantes :

Spécification de sortie	Quels résultats doit-on obtenir ? Selon quel format ?

Spécification d'entrée	Quelles valeurs doivent être introduites, comment ? Avec quelles contraintes ? (types, valeurs, etc.) Quelle réaction faut-il prévoir en cas d'erreur ? etc.
Spécification de traitement	Quel traitement faut-il effectuer ? Quelles sont les règles de gestion à respecter ? Existe-il des cas particuliers ? Comment doivent-ils être traités ? etc.

3.2 PHASE DE CONCEPTION

S'il est très facile de rédiger un programme de résolution d'équation du premier degré, il s'avère bien plus délicat de concevoir un logiciel de gestion de production. La différence étant sur le plan de la complexité. Le premier est tellement plus simple qu'il peut être écrit directement, tandis que le second ne peut être appréhendé dans son entier par quiconque.

DESCARTES dans son ouvrage « discours sur la méthode » propose de décomposer tout problème en plusieurs problèmes plus petits. Ceux-ci pouvant être décomposés à leur tour en plus petits jusqu'à l'obtention d'un ensemble ordonné de problèmes élémentaires pour lequel la solution est connue. (*Illustrations*)

La programmation structurée s'appuie sur une analyse descendante « Top Down » du problème à traiter. Elle énonce un ensemble de règles destiné à faciliter le développement de toute application informatique :

- Le développement d'un algorithme doit être effectué indépendamment de tout langage de programmation ;
- Le programme doit être construit par niveau successif.

3.3 PHASE DE CODAGE

L'algorithme est traduit dans un langage de programmation (pascal, c, c++, python, pharo, delphi, php, windev , oracle, java, vb,web etc. …). Cela consiste à écrire des codes.

3.4 PHASE DE CORRECTION

La recherche des erreurs est une tâche souvent ingrate car un programme fonctionne rarement à sa première exécution. Elle représente une part très importante du temps de développement d'une application. On distingue trois (03) types d'erreurs :

- **Les erreurs de syntaxe** dues à une mauvaise connaissance du langage utilisé ou plus simplement à des fautes d'inattention. Elles sont détectées lors de la phase de traduction (compilation ou interprétation) ;

- **Les erreurs d'exécution** entrainent un comportement anormal du programme dans certains cas particuliers. Elles proviennent généralement d'un défaut de conception. Leur détection nécessite l'emploi d'un jeu d'essai exhaustif.
- **Les erreurs de logiques** ont pour origine une mauvaise transcription du programme ou plus généralement un mauvais algorithme.

La programmation structurée vise à éliminer ce type d'erreurs et permet aussi un gain considérable de temps.

3.5 PHASE DE MAINTENANCE

Maintenir un programme consiste à le mettre à jour lors de l'apparition de nouveaux besoins. On estime que les services informatiques peuvent consacrer jusqu'à 70% de leur temps à la maintenance de leurs applications existantes.

La programmation modulaire favorise une maintenance aisée pour modifier une partie d'un programme. Il s'suffit de transformer le module concerné sans avoir à changer la structure de l'algorithme tout en entier.

II. LES VARIABLES ALGORITHMIQUES

INTRODUCTION

L'univers des objets d'un traitement est fini. Il est donc possible de le décrire entièrement et sans ambiguïté en décrivant chaque objet. Le traitement d'un objet concerne la valeur de cet objet. Si cette valeur ne peut pas être modifiée, nous parlons de **constante**, sinon nous parlons de **variable**.

1. DEFINITION

Une variable algorithmique est une entité abstraite composée de :

- Un identificateur,
- Une valeur appartenant à un domaine de définition ;

2. NOTION DE VARIABLE

Dans le problème d'addition de deux (02) nombres entiers au paravent, trois objets apparaissent A, B, RESULTAT. Chacun de ces objets est une variable. Le rôle de chacune de ces variables consiste à retenir une valeur particulière significative de l'état du problème dont l'algorithme décrit le comportement.

La valeur retenue par un objet n'est pas forcement la même à tout moment : c'est la seule justification du vocable employé pour désigner ces objets : le mot **variable**. La valeur relative d'une variable est stockée en mémoire centrale dans un ou plusieurs mots (ensemble d'octets). Chaque variable se trouve à une adresse particulière de la mémoire.

3. CARACTERISTIQUES D'UNE VARIABLE

Une variable se caractérise par :

- **Un rôle** : c'est la signification qu'on attribue a la variable,
- **Un identificateur** : il permet d'accéder directement à son contenu sans avoir à utiliser la notion d'adresse qui doit être la plus explicite possible.
- **Un type** : c'est la définition du domaine dans lequel une variable peut puiser sa valeur : le domaine des entiers, des réels, etc. Le type de la variable détermine les opérations que l'on peut appliquer.
- **Une valeur** : c'est le contenu de la variable à un moment donné. Une variable ne possède qu'une valeur en un moment donné. Une variable dont la valeur n'évolue jamais est une **constante**.

On détermine les types élémentaires suivants :

Type d'objet	Valeur exemple	Quelques opérations associées
BOOLEEN	Vrai Faux	Comparaison (=, >, <) Négation (NON), Conjonction (ET), Disjonction (OU)
CARACTERE	« M » ; « ? » ; « 4 »	Comparaison, Conversion (code…)
ENTIER	13 ; 7	Comparaison, Addition (+), Division (/), Division entière (DIV), Congruence (MODULO)
REEL	9,3 ; 2 ; 5.10^3	Comparaison, Addition, Division, Partie entière, Soustraction
CHAINE DE CARACTERES	« Humanité » ; « 149 »	Comparaison, Longueur, Extraction, Concaténation (+)

4. RAPPEL DES TABLES DE VERITE SUR LES TYPES BOOLEEN

A	**B**	**A et B**	**A ou B**	**non A**	**non B**
V	V	V	V	F	F
V	F	F	V	F	V
F	V	F	V	V	F
F	F	F	F	V	V

III. CONVENTIONS SYNTAXIQUES

1. CONVENTIONS

- Une variable ne doit pas commencer par un chiffre ;
- Une variable ne doit pas avoir en son sein un espace ;
- Une variable ne doit pas avoir en son sein un caractère spécial ;
- Une variable ne peut pas être en deux mots auquel cas on les lie par un trait de huit (08) ou un trait de six (06).

Exemple : Cocher les noms des variables qui sont corrects.

total paye	H/3	tête de liste	Moyenne deClasse
Total2	A Mardi4	2Somme	Prix#Total
P1 H20	Amardi4	P1H20	Pourcentage%Terminé

2. DECLARATION DE VARIABLES

La plupart des langages de programmation oblige à déclarer au début tous les éléments qui seront utilisés à la suite en indiquant leur type. Nous adoptons cette règle lors de l'écriture des algorithmes en langage de conception structuré. Ainsi :

- Tous les objets sont décrits en entête de l'algorithme ;
- En même temps que l'objet on déclare son type, ce qui évite toute ambiguïté sur son utilisation ;
- Il est aussi important de donner aux objets des identificateurs parlants.

Exemple de déclaration de variables

```
VAR   nomélève, prénomélève : chaine de caractères
      a, b : entier
      montant, prixttc : reel
      juste : Booléen
```

3. SYNTAXE GENERALE D'UN ALGORITHME

Un algorithme écrit en un langage de conception structuré doit répondre à la syntaxe suivante :

ALGO « identificateur »
{« déclaration »} *
DEBUT
    {« Instructions »} *
FIN

Exemple

ALGO Somme
VAR a, b, s : entier
DEBUT
    Afficher 'Entrer les valeurs'
    Saisir a
    Saisir b
    s := a + b
    Afficher 'Résultat = ', s
FIN

Chapitre 2 : COMPOSANTES ELEMENTAIRES DES ALGORITHMES

I. LES INSTRUCTIONS DE BASE

1. L'AFFECTATION

Elle consiste à affecter une valeur à un objet. Elle permet de valoriser une variable. Cette opération est représentée par une **flèche orientée à gauche** (←) ou par le signe **: =**

Format général

< identificateur> ← < valeur >

Après exécution l'identificateur contient valeur. La valeur peut être soit :

- Une variable de même type ;
- Une constante du type de l'identificateur ;
- Une expression dont l'évaluation produit un résultat du type de l'identificateur.

2. INSTRUCTION D'ENTRÉE ET SORTIE

Nous disposons d'une instruction de saisie qui permet de récupérer une valeur sur une périphérique d'entrée (le clavier) et d'une instruction d'affichage qui permet l'édition d'une valeur sur un périphérique de sortie (imprimante, écran).

Format général

Saisir < identificateur > « Instruction d'entrée » (**Lire** ou **Saisir**)
Afficher < valeur > « Instruction de Sortie » (**Ecrire** ou **Afficher**).

L'identificateur est un objet déclaré. La valeur située derrière l'ordre d'affichage peut être : soit un **identificateur**, **une constante**, **une expression**. Dans le cas d'un dialogue via l'écran et le clavier, il convient d'afficher des messages permettant de faciliter la conversation.

Exemple

```
ALGO Affichage
VAR cycle : caractère
    effectif : entier
DEBUT
  Saisir cycle
  Saisir effectif
  Afficher cycle, ″compte cette année″, effectif, ″ étudiants″
FIN
```

II. LES STRUCTURES CONDITIONNELLES

1. LA STRUCTURE ALTERNATIVE

1.1 STRUCTURE ALTERNATIVE RÉDUITE

Format général

Si < Condition >
    Alors < Action >
FIN Si

L'instruction n'est exécutée que si l'évaluation de la condition ou de l'expression donne un résultat égal à **VRAI**.

1.2 STRUCTURE ALTERNATIVE COMPLÈTE

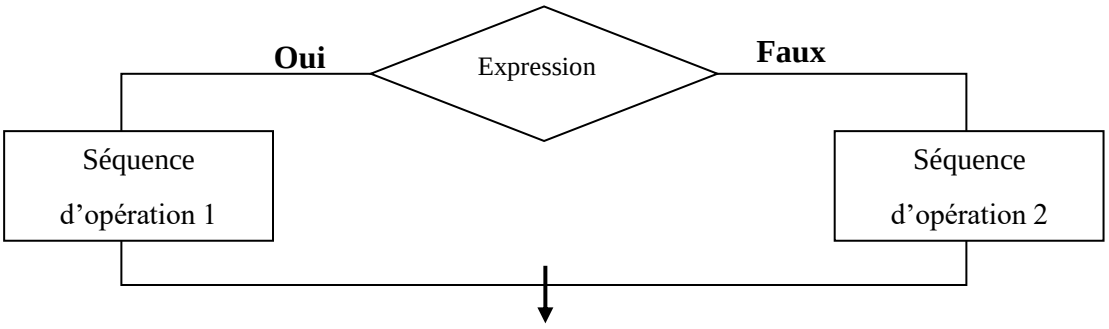
Format général

Si < Condition >
    Alors < Action1 >
Sinon < Action2 >

FIN Si

Lorsque l'évaluation de la condition produit la valeur **VRAI**, l'action 1 est réalisée ; lorsqu'elle produit **FAUX** l'action 2 est réalisée. Action 1 comme action 2 peuvent être soit : **une instruction, un ensemble d'instructions**, **un algorithme**.

1.3 REPRÉSENTATION GÉNÉRALE D'UNE STRUCTURE ALTERNATIVE



On a dans beaucoup de cas une expression logique simple par exemple A > 20.

Mais on peut avoir également des expressions logiques complexes dont la définition générale est indiquée ci-après. Une expression logique est composée :

- D'une part d'**expression élémentaire** liant chacune deux variables par des opérateurs de comparaison (= ; ≠ ; < ; > ; ≤ ; ≥) ;
- D'autre part par **des liens** entre ces expressions élémentaires réalisés par des opérateurs logiques (et ; ou ; non).

1.4 IMBRICATION DE SI

Dans certains cas, l'expression de la condition d'un SI ne suffit pas pour exprimer tous les cas de figure. Nous pouvons alors exprimer un SI après le ALORS d'un SI et ou après le SINON d'un SI. Nous pouvons également mélanger les structures alternatives et conditionnelles.

1.5 STRUCTURE DE CHOIX

La structure du choix permet une présentation plus claire d'un ensemble d'alternatives imbriquées. Plusieurs valeurs peuvent entraîner un même traitement. Nous pouvons alors énumérer ces valeurs en les séparant par des virgules avant de décrire le traitement à entreprendre.

Format général

Suivant < Variable ou expression > **Faire**
ou
Selon que < Variable ou expression > **Faire**

< Valeur 1> : < action 1 >
< Valeur 2> : < action 2 >
< Valeur 3> : < action 3 >
< Valeur n> : < action n >

Sinon < action par défaut >

FIN Suivant

La variable ou l'évaluation de l'expression est comparée aux différentes constantes : valeur « **1** » à valeur « **n** » et les actions entreprises dépendent de cette valeur. Nous disposons d'une action par défaut ou le calcul ou la variable n'est égale à aucune des constantes énumérées.

Exemple :

A partir d'un menu affiché à l'écran, effectuer la somme ou le produit ou la moyenne de 3 nombres. Nous appelons menu l'association d'un séquentiel aux différents choix proposés par un programme.

Résolution

ALGO Menu_Ecran

VAR nbre1, nbre2, nbre3 : réels

choix : entier

DEBUT

(*Saisie des 3 nombres *)
Afficher ″entrer les 3 nombres″
Saisir nbre1, nbre2, nbre3
(* Affichage du menu et saisie du choix *)
Afficher ″1 – pour le produit″
Afficher ″2 – pour la somme″
Afficher ″3 – pour la moyenne″
Afficher ″Faites votre choix″
Saisir choix
Suivant choix **faire**
1 : **Afficher** ″le produit des 3 nombres est″, nbre1 * nbre2 * nbre3
2 : **Afficher** ″la somme des 3 nombres est″, nbre1 + nbre2 + nbre3
3 : **Afficher** ″la moyenne des 3 nombres est″, (nbre1 + nbre2 + nbre3)/3
Sinon Afficher ″cette saisie est incorrecte″
FIN suivant
FIN

1.6 STRUCTURE ALTERNATIVE COMPLEXE : **représentation par une table de décision**

Généralités

La table de décision est un outil qui permet de présenter de façon concise les règles de logique pour décider des actions à exécuter en tenant compte des conditions et de la logique des décisions des problèmes considérés.

Cet outil se présente sous la forme d'un tableau dans lequel on distingue :

- **La partie des conditions (la souche) ;**
- **La partie des actions (la souche) ;**
- **La partie des règles.**

On peut distinguer quatre (04) zones dans le tableau :

- **En haut à gauche :** libellé des conditions ;
- **En bas en gauche :** libellé des actions, ces deux premières zones constituent la souche ;
- **En haut en droite :** combinaison possible des conditions ;
- **En bas en droite :** désignation des conditions à exécuter.

Le principe consiste donc à associer entre elles les conditions de toutes les manières possibles pour envisager l'ensemble des situations ou cas réalisables et indiquer en fin de chaque cas la ou les actions à accomplir.

Le nombre de cas possible peut être déterminé mathématiquement si l'on suppose par exemple que les conditions sont binaires (**oui** ou **non**) et s'il y a **n** condition(s), il y a $\mathbf{2^n}$ cas possibles.

Exemple :

Traduisons sous forme de table de décision, la logique qui permettra à Léon de décider de ses activités dimanche prochain. Le texte à traduire est le suivant :

Les activités de dimanche prochain de Léon dépendront de deux conditions : le temps et la qualité des programmes de la télévision. Ses activités seront la pêche, la télévision ou le cinéma. Il ira à la pêche s'il fait beau et ce, que les programmes de la télévision soient intéressants ou pas. Il regardera la télé s'il ne fait pas beau et si les programmes sont intéressants. Il ira au cinéma s'il ne fait pas beau et si les programmes de la télé ne l'intéressent pas. Léon passera son dimanche en famille s'il reste à la maison, c'est à dire s'il regarde la télé.

Résolution

La table de décision suivante traduit la logique de décisions de Léon quant à ses activités le dimanche :

Règles de gestion

			R1	R2	R3	R4
CONDITIONS	C1	**Fait – il beau** ?	O	O	N	N
	C2	**Les programmes de la télévision sont – ils intéressants** ?	O	N	O	N
ACTIONS	A1	Léon va à la pêche	×	×		
	A2	Léon regarde la télévision			×	
	A3	Léon va au cinéma				×
	A4	Léon reste en famille			×	

Légende : O veut dire Oui
N ″ ″ Non
X ″ ″ Action à exécuter

Présentation des tables de décision

Les tables de décision sont présentées en deux modes :

- **La présentation horizontale :**
  Les conditions et les actions sont écrites horizontalement ; Les règles ou cas sont lus verticalement.
- **La présentation verticale :**
  Les conditions et les actions sont écrites verticalement ; les règles sont lues horizontalement.

Principaux domaines d'utilisation des tables de décision en informatique

- Les tables de décision sont surtout utilisées pour traduire les textes administratifs, législatifs qui décrivent les règles de gestion, de procédure à automatiser. Elles permettent de clarifier les textes souvent très difficiles à comprendre et de déceler les anomalies et les oublis.
  Les utilisateurs eux-mêmes peuvent traduire les règles de gestion en table de décision ; cependant, ce travail est souvent réalisé par les analystes.
- Durant l'étude d'une application les tables de décision peuvent être utilisées chaque fois qu'une logique de décision assez complexe est à définir et à préciser avant d'être programmée : définition d'une transaction clavier, définition de la logique de contrôle, définition de la logique d'une cinématique de fichier.
- Les tables de décision peuvent aussi servir à préciser les consignes de travail du personnel qui utilisera les procédures d'une application informatique.

Table de décision à entrée limitée

Ce sont des tables où :

- Les cases situées en regard des conditions sont remplies avec des **Oui**, des **Non** et des signes (=). **Le signe (=) placé dans une case signifie que la condition située en regard de cette case n'intervient pas dans la règle considérée.**
- Les cases situées en regard des actions ne contiennent que des croix.

La table de décision en exemple sur les activités de dimanche de Léon est une table à entrée limitée. La table est dite « **limitée** » lorsque chaque condition et chaque action sont entièrement décrites dans la souche.

La table de décision sur les activités de dimanche peut se représenter comme suit :

	cas1	cas2	cas3
Fait – il beau ?	O	N	N
Les programmes de la télévision sont – ils intéressants ?	=	O	N
Léon va à la pêche	X		
Léon regarde la télévision		X	
Léon va au cinéma			X
Léon reste en famille		X	

Exercice d'application

Considérons les règles pour un gestionnaire de stock qui a une commande à satisfaire :

Condition 1 : qté à livrer < = qté en stock.
Si cette condition est réalisée, on exécute l'action 1 : livrer la commande.

Condition 2 : qté en stock – qté à livrer < = au seuil minimum.
Si cette condition est réalisée, on exécute l'action 2 : ordonner le réapprovisionnement.

Question : Comment peuvent être représentées les décisions du gestionnaire de stock face à l'arrivée de commande ?

Résolution

	cas1	cas2	cas3
Qté à livrer ≤ en stock	O	O	N
Qté après livraison ≤ au seuil minimum	O	N	–
Livrer la commande	X	X	–
Ordonner le réapprovisionnement	X	–	X

2. LA STRUCTURE RÉPÉTITIVE

Définition

La structure répétitive permet la répétition d'une action jusqu'à ce qu'une condition soit vérifiée. Elle ressemble à la structure itérative mais la différence est que la condition exprimée permet l'arrêt du traitement. De plus la condition n'est testée qu'après une première exécution de l'action définie. Les instructions sont exécutées au moins une fois. La structure répétitive ne nécessite pas comme la structure itérative une action préalable au test hors de la boucle du fait que la condition est évaluée en fin de boucle.

Format général

Répéter

{< Instruction >}

Jusqu'à < Condition >

Exemple

Ecrire l'algorithme permettant de calculer l'ancienneté d'un employé. L'année d'embauche sera saisie au clavier et le résultat affiché à l'écran. Le calcul sera effectué au tant de fois qu'on le désire.

Méthode ou analyse

L'algorithme sera en 3 phases :

- Saisie de l 'année d'embauche et de l'année de calcul ;
- Calcul de l'ancienneté ;
- Saisie de la réponse à la question de continuer ou pas ;

4 objets peuvent - être utilisés :

- Année_emb de type entier qui reçoit l'année d'embauche
- Année_cal de type entier qui reçoit l'année de calcul
- Ancienneté de type entier reçoit le résultat du calcul de l'ancienneté
- Réponse : type de chaîne de caractère reçoit la réponse à la question

Résolution

L'ancienneté est obtenue en faisant la différence entre l'année de calcul et l'année d'embauche. Le traitement est arrêté lorsque la réponse à la question est non.

```
ALGO calcul_anciennete
VAR ancienneté, année_emb, année_cal : entier
      réponse : caractère
DEBUT
    REPETER
        AFFICHER « Entrer l'année d'embauche »
        SAISIR année_emb
        AFFICHER « Entrer l'année de calcul »
        SAISIR année_cal
        (* calcul de l'ancienneté *)
        ancienneté ⟵ année_cal – année_emb
        AFFICHER « L'ancienneté est », ancienneté
        AFFICHER « Y a-t-il d'autres employés à traiter ? O / N »
        SAISIR réponse
    JUSQU'A réponse = 'N'
FIN
```

3. LA STRUCTURE ITÉRATIVE

Définition

La structure itérative permet la répétition d'une ou de plusieurs actions tant qu'une condition est satisfaite.

La condition est testée avant la première exécution de l'action définie. La condition exprimée après l'instruction TANT QUE permet l'exécution de l'action définie.

Une structure itérative est utilisée lorsque l'action peut – être exécutée 0 ou n fois. On prend la proposition contraire pour rechercher la condition d'arrêt du traitement.

Format général

TANT QUE < Expression logique >
    { < Instruction > }

FIN TANT QUE

Exemple

Ecrire l'algorithme permettant de calculer la moyenne des notes d'un élève en affichant les notes saisies à l'écran.

Méthode ou analyse

L'algorithme sera en 3 phases :

- Initialisation d'un compteur de notes à 0
- Saisir les notes, les compter, les afficher et les cumuler.
- Editer la moyenne et le nombre de notes saisies

Résolution

4 objets peuvent - être utilisés :

- Nbnote : Numérique contenant le nombre de notes
- Note : Numérique contenant la note saisie au clavier
- Cumul : Numérique contenant la somme des notes saisies
- Moyenne : Numérique contenant la moyenne des notes saisies.

```
ALGO Moyenne
VAR note, cumul, moyenne : réel
      Nbnote : entier
DEBUT
   (* Saisie de la 1ère note *)
   AFFICHER ″Entrer une note (ou 99 pour arrêt)″
   SAISIR note
   (* Initialisation du compteur de note *)
   nbnote ⟵ 0
    (* Initialisation du compteur de cumul *)
   Cumul ⟵ 0
   (* début de la boucle *)
   TANT QUE NOTE < > 99
   nbnote ⟵ nbnote + 1
   cumul ⟵ cumul + note
   SAISIR note
   FIN TANT QUE
   AFFICHER ″L'élève a ″, nbnote, ″ notes″
   moyenne ⟵ cumul / nbnote
   AFFICHER ″Le total des notes est ″, cumul
   AFFICHER ″La moyenne des notes est″, moyenne
FIN
```

4. RÉSUMÉ DES STRUCTURES RÉPÉTITIVES

4.1 LA STRUCTURE RÉPÉTITIVE **TANT QUE… FIN TANT QUE**

Format général

TANT QUE < Expression logique >
{ < Instruction > }

FIN TANT QUE

On teste d'abord si l'expression logique est **vraie**. Dans ce cas, on exécute les instructions puis on boucle de nouveau sur le test d'expression logique. Par contre si l'expression logique est **fausse**, le programme poursuit son exécution après FIN TANT QUE.

Remarque :

Le test est effectué avant l'exécution du bloc ; il est donc possible de n'avoir aucune exécution du bloc d'instruction.
Comme il est conseillé de ne pas boucler indéfiniment, une instruction au moins du ″bloc instruction″ doit agir sur l'expression logique pour la rendre fausse.
″**Le bloc instruction**″ peut contenir des structures alternatives et des structures répétitives imbriquées.

Exemple

Ecrire l'algorithme permettant de calculer le produit de deux (02) entiers positifs en n'utilisant que l'opération d'addition.

Méthode ou analyse
Produit ⟵ A*B ⇔ Produit ⟵ A+A+A+…+A
L'addition est effectuée B fois.
La valeur de la variable B est stockée dans une variable décrémentée à chaque boucle ; boucle dans laquelle on additionne la valeur de A à la valeur présente de produit dont la zone est initialisée à zéro (o).

Résolution

ALGO multiplication
VAR Produit A, B, Compteur : entier
DEBUT
Produit : = 0
Afficher ″ Veuillez indiquer 2 entiers positifs″
Saisir A, B
Compteur ⟵ B
Tant que Compteur< > 0
Produit : = Produit +A
Compteur : = Compteur –1
FIN Tant que

Afficher ″le produit ″, A ″ par ″, B, ″ égal à ″ ; Produit
FIN

4.2 LA STRUCTURE RÉPÉTITIVE **RÉPÉTER … JUSQU'A**

Format général

Répéter
{< Instruction >}

Jusqu'à < Expression logique >

La sémantique est la suivante :

- Le bloc <instructions>est exécuté.
- L'<expression logique> est testée :
  Dans le cas où elle est égale à faux, on recommence au point 1. Dans le cas où elle est égale à vrai, le programme poursuit son exécution après l'instruction « Jusqu'à »

Remarques

Avec cette forme de structure répétitive, le bloc d'instructions est exécuté au moins une fois ! Cela peut être source d'erreurs dans certains cas.
La condition exprimée par l'expression logique est l'expression contraire de celle exprimée dans la structure répétitive de type « Tant que ».

Exemple

Ainsi pour passer de la structure **Tant Que… Fin Tant que** à la structure **Répéter…Jusqu'à** il suffit d'inverser la condition d'arrêt. (Compteur <>0 devient Compteur = 0).

Résolution

ALGO multiplication
VAR Produit A, B, Compteur : entier
DEBUT
Produit : = 0
Afficher ″ Veuillez indiquer 2 entiers positifs″
Saisir A, B
Compteur ⟵ B
Répéter
Produit : = Produit +A
Compteur : = Compteur –1
Jusqu'à Compteur = 0
Afficher ″le produit ″, A , ″ par ″, B , ″ égal à ″ , Produit
FIN

Mais dans le cas où B est inférieur ou égal à zéro l'algorithme boucle car Les instructions comprises dans la boucle sont exécutées au moins une Fois ! D'où la nécessité de forcer

l'utilisateur à introduire deux valeurs strictement positives ! La structure répétitive Répéter… jusqu'à permet d'ailleurs de gérer plus facilement le contrôle sur les données à saisir puisqu'un seul message suffit. Elle est très souvent utilisée pour contrôler des données saisies.

ALGO multiplication
VAR Produit A, B, Compteur : entier
DEBUT
    Produit : = 0
    Répéter
        Afficher ″ Veuillez indiquer 2 entiers positifs″
        Saisir A, B
    Jusqu'à (A > 0 **et** B > 0)
    Compteur ⟵B
    Répéter
        Produit : = Produit +A
        Compteur : = Compteur –1
    Jusqu'à Compteur = 0
    Afficher ″le produit″, A, ″ par ″, B , ″ égal à ″, Produit
FIN

4.3 LA STRUCTURE RÉPÉTITIVE POUR … FIN POUR

Cette action permet de répéter l'exécution d'une ou plusieurs instructions un nombre de fois déterminé. Le nombre des répétitions est connu ou déterminable avant la première exécution de l'action concernée. L'exécution est conditionnée par l'observation d'une variable de test appelée indice de boucle ou compteur, qui est initialisée puis incrémentée ou décrémentée de façon implicite, après chaque passage dans la boucle. La condition porte uniquement sur le nombre de répétitions.

Format général

Pour <identificateur> **de** <expression début> **à** <expression fin> (**pas de** <expression pas>)
    {<Instructions>}
Fin Pour

<Identificateur> est une variable de type entier. <Expression début>, <expression fin> et <expression pas> sont des expressions numériques de type entier.

La sémantique est la suivante :

- <identificateur>est une variable assimilée à un compteur qui est automatiquement augmenté ou diminué en fonction de <expression pas> ;
- Au début, <identificateur> prend la valeur de <expression début> ;
- Le bloc d'instructions est exécuté jusqu'à ce que <identificateur> prenne la valeur de <expression fin> ;
- La valeur <identificateur> évolue de < expression début> jusqu'à <expression fin>en fonction de <expression pas> qui par défaut est égale à 1.

Exemple : Affichage de la table de multiplication de 10

Cette action permet de répéter l'exécution d'une ou plusieurs instructions un nombre de fois déterminé. Le nombre des répétitions est connu ou déterminable avant la première exécution de l'action concernée. L'exécution est conditionnée par l'observation d'une variable de test appelée indice de boucle ou compteur, qui est initialisée puis incrémentée ou décrémentée de façon implicite, après chaque passage dans la boucle. La condition porte uniquement sur le nombre de répétitions.

```
ALGO table_de_multiplication_de_dix
VAR k : entier
DEBUT
    Afficher ″ Table de multiplication de 10 ″
    Pour k de 1 à 10
        Afficher k, ″ * 10 = ″, k*10
    Fin pour
FIN
```

```
1*10 = 10
2*10 = 20
3*10 = 30
…
8*10 = 80
9*10 = 90
10*10 = 100
```

Dans cet algorithme, le pas de progression de l'indice k est implicitement 1.

Le même algorithme peut s'écrire avec une structure **Tant que** ou une structure **Répéter… jusqu'à**. La gestion de l'indice étant à la charge du concepteur de l'algorithme.

```
ALGO table_de_multiplication_de_dix
VAR k : entier
DEBUT
    k : = 1
    Afficher  ″ Table de multiplication de 10 ″
    Tant que k <=10 Faire
        Afficher k, ″ * 10 = ″, k*10
        k := k + 1
    Fin Tant que
FIN
```

Il en est de même avec une structure **répéter… jusqu'à :**

```
ALGO table_de_multiplication_de_dix
VAR k : entier
DEBUT
    k : = 1
    Afficher  ″ Table de multiplication de 10 ″
    Répéter
        Afficher k, ″ * 10 = ″, k*10
        k := k + 1
    Jusqu'à k > 10
FIN
```

Exemple avec utilisation d'un pas prédéterminé

Affichage des nombres pairs allant de 2 à 100

- Avec une structure Pour

```
ALGO Nombres_pairs
VAR k : entier
DEBUT
    Afficher ″ liste des nombres pairs de 2 à 100 ″
    Pour k de 2 à 100 pas de 2
            Afficher k
    Fin pour
FIN
```

- Avec une structure Tant que

```
ALGO Nombres_pairs
VAR k: entier
DEBUT
    k : = 2
    Afficher  ″ liste des nombres pairs de 2 à 100 ″
    Tant que k <=100 Faire
            Afficher k, ″ * 10 = ″, k*10
            k := k + 2
    Fin Tant que
FIN
```

- Avec une structure Répéter…jusqu'à

```
ALGO Nombres_pairs
VAR k: entier
DEBUT
    k : = 2
    Afficher  ″ liste des nombres pairs de 2 à 100 ″
    Répéter
            Afficher k
            k := k + 2
    Jusqu'à k > 100
FIN
```

5. RÉSUMÉ

5.1 CONTENU D'UN ALGORITHME

Un algorithme est constitué :

- Des opérations (lire, calculer, écrire) ;
- Un ordre de succession des opérations ;
- Des conditions provoquant l'exécution ou la non-exécution de telle opération et par des itératives (répétition de plusieurs exécutions du traitement) ou ″boucle.

5.2 REPRÉSENTATION D'UN ALGORITHME

- Représentation par un organigramme

La sémantique est la suivante :

- On représente chaque opération par un symbole spécifique, généralement par un 
- L'ordre de succession est représenté par des flèches 
- Les conditions sont représentées par un symbole spécifique, généralement un 
- Enfin pour la clarté de la représentation, on indique le début et la fin de l'algo par un symbole spécifique (rectangle arrondi  ou ellipse )

- Représentation par un langage structuré

La représentation des opérations et de leur ordre se fait en écrivant leurs contenus de bas en haut. Le programme est identifié par un nom ; les variables sont déclarées à partir du préfixe Var, les opérations sont contenues entre début et fin.

5.3 REPRÉSENTATION DES DONNÉES ET DES CALCULS SUR LES DONNÉES

On aura intérêt pour enlever la représentation d'un algorithme à coder les noms des données soit par des mnémoniques soit par des lettres éventuellement indicées.

Exemple :

Désignation	Code
Remise	Remise
Quantité commandée	Quantité_commandée

Chapitre 3 : LES TABLEAUX ET LES FICHIERS

I. LA GESTION DES TABLEAUX

INTRODUCTION

Tous les algorithmes et programmes présentés jusqu'ici ont manipulé des variables simples à un instant donné, la variable ne possédant qu'une seule valeur. Certains problèmes nécessitent l'utilisation de variables structurées possédant simultanément plusieurs variables.

L'algorithme ci-dessous permet de saisir n nombres et d'en calculer la somme et la moyenne.

ALGO Moyenne
VAR somme, moyenne, note : réel
    Nombre-notes, compteur : entier
DEBUT
Afficher « entrer le nombre de notes à saisir »
Saisir Nombre-notes
Somme : = 0
Pour compteur **de** 1 **à** nombre-notes
    Afficher ″ Entrer une note″
    Saisir note
    somme : = somme + note
Fin pour
moyenne : = somme / nombre-notes
Afficher ″ la somme est de ″, somme,
Afficher ″ la moyenne est de ″, moyenne
FIN

Admettons que le nombre de notes à saisir soit 4 et que l'on souhaite conserver en mémoire l'ensemble des notes saisies pour les afficher ; on ne peut plus se contenter d'une seule variable nombre et l'on est alors amené à écrire l'algorithme suivant :

ALGO Moyenne

VAR Somme, Moyenne, Note1, Note2, Note3, Note4 : réels

DEBUT

Somme : =0

Afficher ″ entrer une 1ère note ″

Saisir note1

Afficher ″ entrer une 2è note″

Saisir note2

Afficher ″ entrer une 3è note″

Saisir note3

Afficher ″ entrer une 4° note ″

Saisir note4

Somme := note1 + note2 + note3 + note4

Moyenne := somme/4

Afficher ″la somme des notes ″, note1, note2, note3, note4, ″ est de ″, Somme ,

″ et la moyenne est de ″, Moyenne

FIN

DÉFINITION

Un tableau est une structure de données ayant une dimension et un type déterminé, qui permet de stocker en mémoire plusieurs valeurs de même type. Ces valeurs sont appelées éléments du tableau. Chacun de ces éléments est repéré par un indice indiquant sa position relative par rapport au début du tableau.

Dimension : le nombre d'éléments que peut contenir le tableau détermine la dimension du tableau.

Type : désigne la nature des valeurs stockées dans le tableau (entier, caractère, chaîne, réel, …).

Elément : façon de désigner une valeur d'un tableau.

Indice : un indice est un entier naturel dont les valeurs varient de 1 à la dimension du tableau. A chaque valeur de l'indice correspond une seule valeur dans le tableau.

Représentation logique

Tableau T de type entier. Dimension = n

882	288	82	…	896	86	8
1	2	3		n-2	n-1	n

Le 1er élément du tableau est noté T(1)

Le 2e élément du tableau est noté T(2)

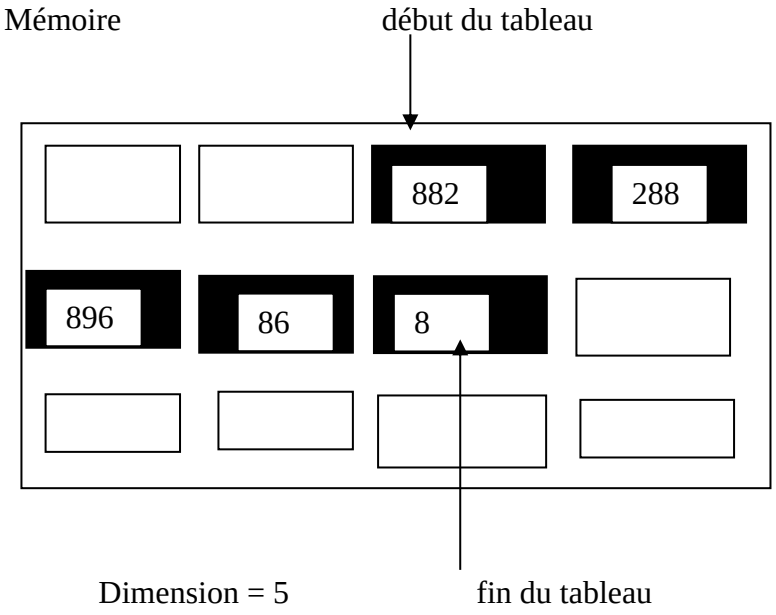
…

Le n-1e élément du tableau est noté T(n-1)

Le ne élément du tableau est noté T(n)

Représentation en mémoire

Un tableau est stocké en mémoire où il occupe une suite d'emplacements successifs.



Syntaxe de déclaration d'un tableau

Déclaration de tableau : := <identificateur><dimension> : [tableau de] <type>

Dimension : := [<entier>]

Type : := entier | réel | car | booléen

La dimension doit être connue à l'avance. Le tableau peut ne pas être entièrement rempli, mais il ne peut en aucun cas contenir plus d'éléments que le nombre déterminé lors de la déclaration. L'indice d'un tableau de dimension **n** est un entier déclaré indépendamment variant de 1 à **n**.

Exemples

Nom_élèves (32) : tableau de chaîne de caractères
Représente un tableau de type chaîne de caractères permettant de stocker les 32 noms des étudiants de la classe.

Moyennes (11) : tableau de réel
Représente un tableau de type réel permettant de stocker 11 moyennes. Dans la déclaration d'un tableau, l'expression « tableau de » est facultative.

II. LES TABLEAUX À UNE DIMENSION

1. DÉFINITION

Un tableau est dit à une dimension lorsqu'un seul indice suffit à adresser les éléments du tableau. L'utilisation des éléments d'un tableau ne peut se faire qu'élément par élément. L'accès à un élément se fait en indiquant directement la position relative de l'élément par rapport au début du tableau :

Exemples

T (8) pour accéder au 8e élément du tableau.

T (k) pour accéder au k^{e} élément du tableau.

2. GESTION DES ÉLÉMENTS D'UN TABLEAU

Un élément d'un tableau est considéré comme une variable à part entière et peut par conséquent être utilisé lors des instructions d'entrée-sortie, d'affectation, mais aussi dans toutes les expressions numériques, conditionnelles, …

2.1 EXERCICE : Ecrire l'algorithme permettant de résoudre le problème au I.1

```
ALGO Moyenne
VAR Somme, Moyenne, Tnotes (100) : réel
    Nombre_notes, Compteur : entier
DEBUT
    Afficher ″entrer le nombre de notes à saisir (<100) ″
    Saisir nombre_notes
    Somme := 0
    Pour Compteur de 1 à Nombre_notes
        Afficher  ″entrer une note ″
        Saisir Tnotes ( Compteur)
        Somme : = somme + Tnotes (Compteur)
    Fin pour
    Moyenne := somme/nombre_notes
    Afficher ″voici l'ensemble des notes de l'élève″
    Pour compteur de 1 à nombre_notes
        Afficher ″la″, compteur, ″ ère/ème note est: ″ Tnotes (Compteur)
    Fin pour
```

Afficher « la somme est de », somme

Afficher « la moyenne est de », moyenne

FIN

Quel que soit le nombre de notes saisies, la taille de l'algorithme n'augmente plus !

Conclusion :

Grâce à la notion de tableau, l'on peut saisir et conserver les notes dans un tableau en vue d'un affichage ultérieur, et ce, en conservant l'algorithme d'origine quel que soit le nombre de notes.

2.2 EXERCICE : Rechercher un élément dans un tableau

- Recherche à l'aide de la position relative de l'élément

ALGO Moyenne
VAR Tnotes (100) : réels
Nombrenotes, Compteur, Position : entier
DEBUT
Afficher ″ Entrer le nombre de notes à saisir (<100) ″
Saisir Nombrenotes
Pour Compteur de 1 à Nombrenotes
Afficher ″ entrer une note : ″
Saisir Tnotes (compteur)
Fin pour
Afficher ″ Donnez la position de l'élément recherché″
Saisir Position
Afficher ″ la valeur est : ″, Tnotes (Position)
FIN

- Recherche à l'aide de la valeur d'un élément (on arrête à la première valeur identique rencontrée)

ALGO Moyenne

VAR Tnotes (100), ValeurRecherchée : réel

Nombrenotes, compteur : entier
Trouve : booléen

DEBUT

Afficher ″ entrer le nombre de notes à saisir (<100) ″

Saisir nombrenotes

Pour Compteur de 1 à Nombrenotes

Afficher ″ entrer une note : ″ ,

Saisir Tnotes (Compteur)

Fin pour

Compteur := 1

Trouve := faux

Afficher ″ donnez la valeur de l'élément recherché ″ ,

Saisir ValeurRecherchée

Tant que (compteur <= Nombrenotes et Non Trouvé) faire

Si Tnotes (compteur) = valeurRrecherchée alors

Afficher ″ l'élément est en ″ , Compteur, ″ position″

Trouvé := vrai

Fin si

Compteur := Compteur + 1

Fin tant que

Si non Trouve **alors**

Afficher ″ l'élément n'est pas dans le tableau″

Fin Si

FIN

Remarque : en cas d'éléments identiques, cet algorithme renvoie la position du premier doublon

2.3 EXERCICE : Rechercher l'élément le plus grand

Objectif
Soit un tableau T de n entiers, donner la valeur du plus grand élément de T ainsi que l'indice de la première occurrence de ce maximum.

Méthode :
L'idée est d'initialiser MAXIMUM avec la valeur du premier élément de T puis de le comparer successivement avec chaque élément en affectant la valeur de ce dernier à MAXIMUM lorsqu'il est plus grand.

- ***En parcourant le tableau de gauche à droite*** ***(sens normal)***

ALGO Moyenne
VAR Tnotes(100), Maximum : réel

Nombre-notes, Compteur, Position : entier

DEBUT

Afficher ″ entrer le nombre de notes à saisir (<100) ″

Saisir Nombre-notes

Pour compteur de 1 à nombre-notes

Afficher ″entrer une note : ″

Saisir Tnotes (compteur)

Fin pour

Maximum : = Tnotes(1)

Position := 1

Pour Compteur de 2 à nombre-notes

Si Maximum < Tnotes (Compteur) alors

Position := compteur

Maximum := Tnotes (Compteur)

Fin si

Fin pour

Afficher ″ le maximum est ″, maximum, ″ et se trouve à la position ″, position

FIN

- ***En parcourant le tableau de droite à gauche*** ***(absolument autorisé)***

ALGO Moyenne
VAR Tnotes(100), Maximum : réel

nombre-notes, compteur, position : entier

DEBUT

Afficher ″ entrer le nombre de notes à saisir (<100) ″

Saisir nombre-notes

Pour compteur de 1 à nombre-notes

Afficher ″entrer une note : ″

Saisir Tnotes (compteur)

Fin pour

Maximum : = Tnotes(1)

position := 1

Pour Compteur de 2 à nombre-notes

Si Maximum < Tnotes (Compteur) alors

position := compteur
Maximum := Tnotes (compteur)

Fin si

Fin pour

Afficher ″ le maximum est ″, Maximum, ″ et se trouve à la position ″, position

FIN

ALGO Moyenne

VAR Tnotes (100), Maximum : réel

Nombre-notes, compteur, Position : entier

DEBUT

Afficher ″entrer le nombre de notes à saisir (<100) ″

Saisir nombre-notes

Pour compteur d**e** 1 **à** Nombre-notes

Afficher ″ entrer une note ″

Saisir Tnotes (Compteur)

Fin pour

Maximum **:= Tnotes (1)**

Position := 1

Pour Compteur **de** Nombre-notes **à** 2 **pas de** –1

Si Maximum < Tnotes (Compteur) **alors**

Position := Compteur

Maximum := Tnotes (Compteur)

Fin si

Fin pour

Afficher ″le maximum est ″, Maximum, ″ et se trouve à la position ″, Position

FIN

III. LES TABLEAUX À DEUX DIMENSIONS

1. ACCÈS ALÉATOIRE

L'accès à un élément de tableau à 2 dimensions s'effectue obligatoirement en donnant la ligne et la colonne où se trouve l'élément.

T[2,10] permet : d'accéder à l'élément situé à la 2ème ligne et dans la 10ième colonne.
T[11,2] permet d'accéder à l'élément situé à la 11ième ligne et dans 2ième colonne.

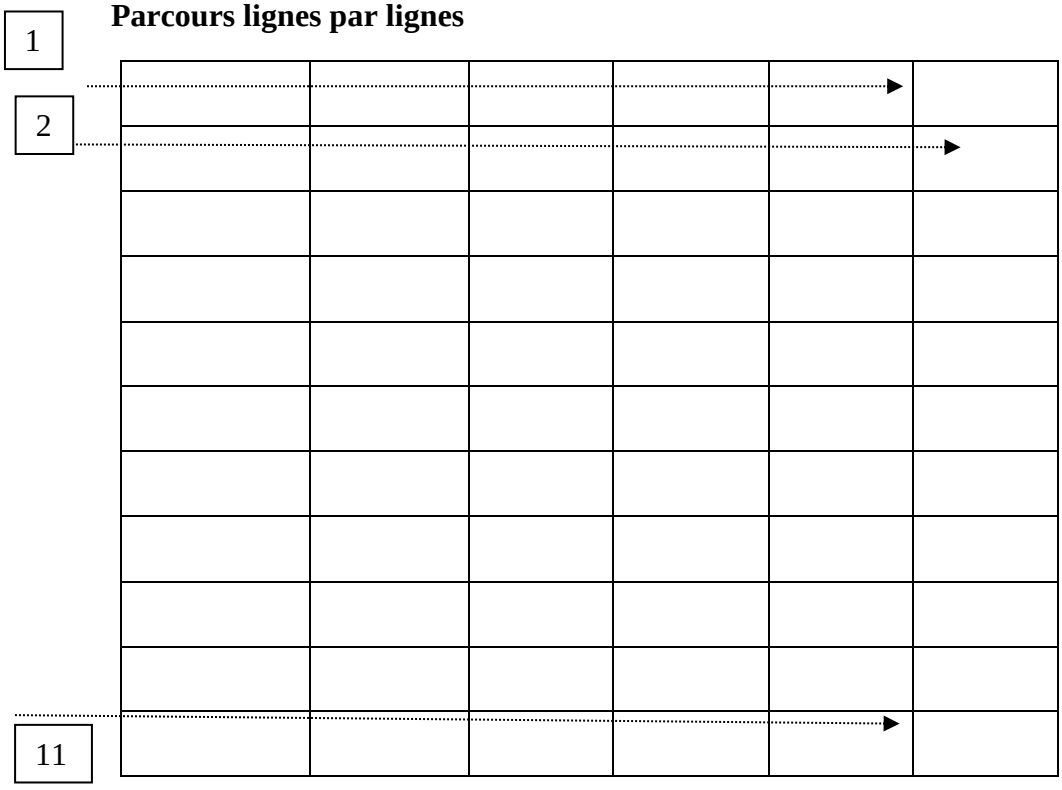
Le 1er indice indique la ligne, le 2nd la colonne

2. MÉTHODE DE PARCOURS

En dehors de l'accès aléatoire à un ou plusieurs éléments dans un tableau, on est souvent amené lors des opérations de saisie, d'affichage, de calcul etc.…à parcourir le tableau dans sa totalité.

Le parcours peut alors s'effectuer ligne par ligne ou bien colonne par colonne.

Parcours lignes par lignes



L'algorithme suivant permet la saisie des notes dans un tableau. Les notes sont saisies élève par élève.

ALGO moyennes_eleves

VAR Tnotes[11,5] : tableau de réel

Nombre-élèves, Nombre-matières, ligne, colonne : entier

DEBUT

Afficher ″ Entrer le nombre d'élèves de la classe (<11) ″

Saisir Nombre-élèves

Afficher ″Entrer le nombre de notes à saisir par élève (< 5) ″

Saisir Nombre-matières

Pour ligne **de** 1 **à** Nombre-élèves

Pour colonne **de** 1 **à** Nombre-matières

Afficher ″Entrer un nombre″

Saisir Tnotes [ligne colonne]

Fin Pour

Fin Pour

FIN

Une fois les notes saisies, les moyennes par élève peuvent être calculées à l'aide d'un parcours ligne par ligne.

…

POUR ligne de 1 à nombre élèves

/ * initialisation de la moyenne de l'élève * /

tnotes (ligne,6) : = 0

POUR colonne de 1 à nombre matières

/ * calcul somme */

tnotes (ligne,6) := tnotes (ligne,6) + tnotes(ligne, colonne)

FIN POUR

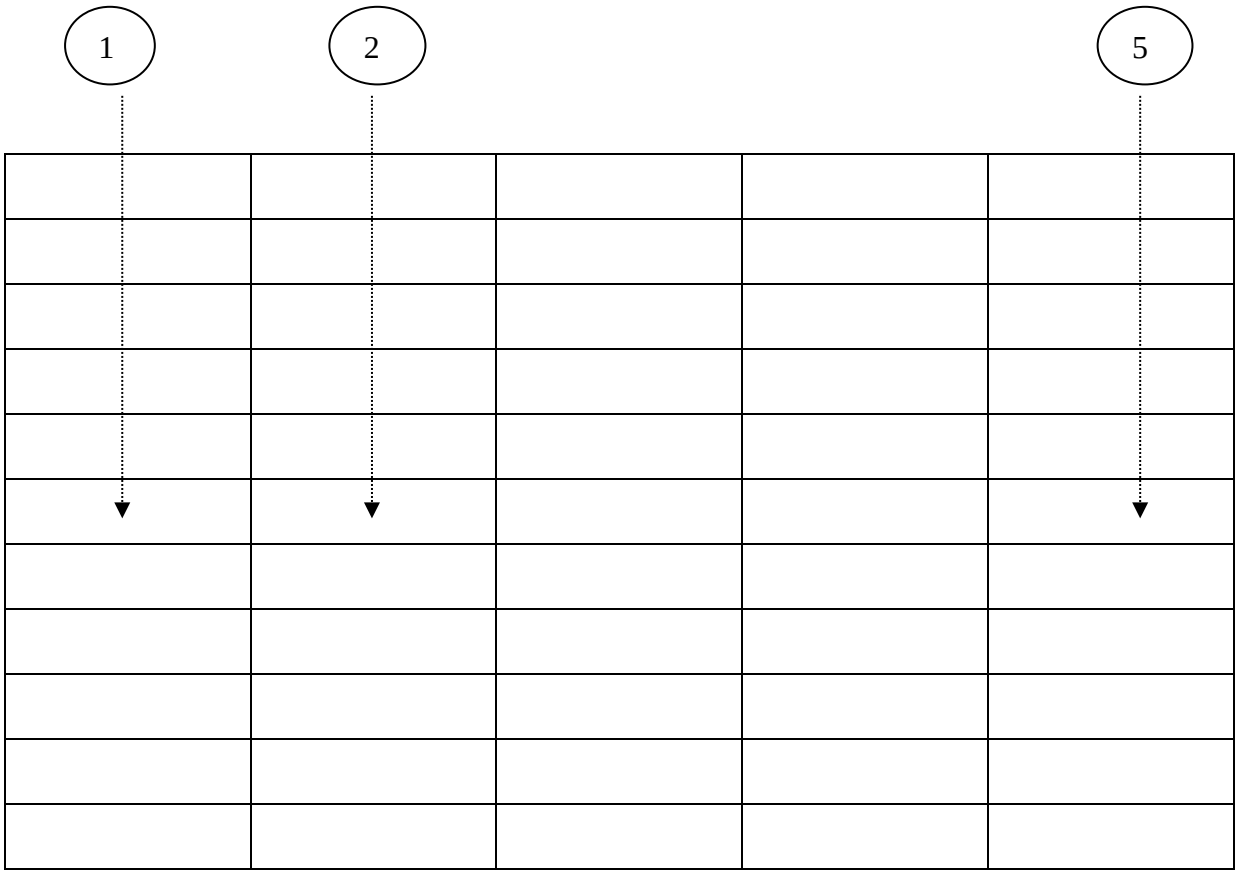
/ * calcul moyenne * /

tnotes(ligne, 6) := tnotes(ligne,6) / nombre notes

FIN POUR

…

Parcours colonnes par colonnes



Le parcours colonne par colonne permet de calculer par exemple la moyenne par matière ainsi que la moyenne générale.

IV. OPÉRATIONS DE TRI SUR LES TABLEAUX

(Cf. les différents exercices programmés à cet effet)

V. NOTION DE FICHIER

INTRODUCTION

Toutes les données utilisées jusqu'à maintenant dans les algorithmes étaient stockées dans des variables en mémoires centrale et n'étaient de ce fait accessible que le temps d'exécution du programme.

La mémoire centrale d'un ordinateur étant par définition volatile, il n'est donc pas envisageable de se limiter à la simple notion de variable pour pouvoir conserver de façon pérenne des données importantes.
Les données relatives à l'activité d'une entreprise (données relatives aux clients, aux fournisseurs, aux produits, aux commandes …) doivent pouvoir être mémorisées de façon durable pour pouvoir être réutilisées à la demande. Ce besoin de conservation de l'information passe par l'utilisation de support non volatile (disque, bande magnétique…) et nécessite de structurer l'information en fichier.

1.DEFINITION

Un fichier est une collection d'informations de même nature ayant trait à un même sujet et se trouvant sur un support quelconque (bande ou caste magnétique, disque ou disquette magnétique, cd-rom, etc.). Ainsi les données relatives aux clients seront regroupées dans un fichier « client », celles relatives aux fournisseurs dans un fichier « fournisseur », etc.

Le choix du support revêt une importance, et conditionne fortement les opérations qu'ils seraient possibles de réaliser sur un fichier. La capacité du support (de quelques Ko à plusieurs Go) détermine la taille maximale du fichier. Le temps d'accès moyen (de quelques millisecondes à plusieurs minutes) conditionne le confort d'accès à l'information.

2.CARACTERISTIQUE D'UN FICHIER

2.1. CARACTERISTIQUES FONCTIONNELLES D'UN FICHIER

- **Volume** : détermine la place occupée en caractères par le fichier ;
- **Taux de renouvèlement** : c'est le nombre relatif d'articles nouveaux qui apparaissent en un an.
- **Taux d'accroissement** : c'est le rapport du nombre absolu d'articles nouveaux au nombre total d'enregistrement pour une période donnée.
- **Fréquence d'utilisation** : c'est le nombre de fois qu'un fichier intervient dans des traitements pour une période donnée.
- **Taux élémentaire de consultation** : c'est le rapport du nombre moyen d'articles consultés en une séance vis-à-vis d'un traitement déterminé.

2.2. CARACTERISTIQUES DES GROUPES DE FICHIERS

a. Support de stockage

Caractérise le support utilisé pour conserver le fichier (papier, support optique, support magnétique, etc.). Le choix du support est fonction des caractéristiques fonctionnelles et induit très fortement les autres caractéristiques techniques. De façon générale, le plus utilisé aujourd'hui est le disque magnétique. La bande magnétique ou ses dérivés sont généralement réservées à l'archivage.

b. Organisation

Caractérise le mode de rangement des articles du fichier et détermine la façon dont les enregistrements seront stockés et retrouvés dans le fichier choisi par l'analyste, l'organisation reste en permanence attachée au fichier. Il s'agit des lors de la caractéristique la plus importante qui conditionne très fortement les opérations autorisées sur le fichier. On distingue les trois (03) organisations suivantes :

- **Organisation séquentielle**

  Les enregistrements d'un fichier séquentiel sont accessibles dans leur ordre de création uniquement. Pour atteindre un enregistrement donné, il faut lire tous ses précédents. Il s'agit d'une organisation impossible à utiliser dans les applications interactives nécessitant un temps d'accès minimum et constant à l'information.

- **Organisation relative**

  Les enregistrements d'un fichier relatif porte chacun un numéro d'ordre relatif à leur emplacement dans le fichier. Le premier enregistrement porte le n° 1, le second le n°2, etc. Ils peuvent être lus et écrits directement grâce à ce numéro d'ordre, ce qui autorise un accès aléatoire a l'information.

- **Organisation séquentielle indexée**

  Dans cette organisation, une structure supplémentaire est associée au fichier. Cette structure appelée « **index** » permet un accès plus rapide aux enregistrements du fichier. Cet index est construit à partir de certains champs de l'enregistrement du fichier. Il peut y avoir plusieurs index associés au fichier.

Les organisations relatives et indexées ne sont possibles que sur les supports autorisant un temps d'accès constant à l'information (disquette ou disque magnétique).

c. Méthode d'accès

Caractéristique étroitement dépendante de l'organisation, elle qualifie la technique utilisée pour accéder aux articles du fichier. Le type d'accès à un fichier est choisi par le programmeur et détermine la façon dont les enregistrements seront recherchés dans le fichier.

- **Accès séquentiel**

Il délivre les enregistrements les uns à la fin des autres :

- Dans leur ordre de création pour un fichier séquentiel ;
- Dans l'ordre des numéros relatifs pour un fichier relatif ;
- Dans l'ordre de l'index pour les fichiers indexés.

- **Accès direct ou aléatoire**

Cet accès n'est possible que sur les fichiers relatifs ou indexés. On accède directement à un enregistrement d'un fichier relatif en connaissant sa position relative dans le fichier. On accède directement à un enregistrement d'un fichier indexé en connaissant la ou les rubriques indexées.

- **Accès dynamique**

Il s'agit d'une combinaison de l'accès directe et de l'accès séquentiel. Combinaison réservée à l'organisation séquentielle indexée.

d. Mode opératoire

On distingue principalement 4 modes de traitement qui sont :

- En lecture uniquement ;
- En écriture uniquement ;
- En mise à jour (lecture et écriture) ;
- En extension.

Cette notion est généralement traduite par le mode d'ouverture du fichier en matière de programmation.

2.3. CARACTERISTIQUES (NON PHYSIQUES) DES FICHIERS

a. Le type

On distingue généralement cinq types de fichiers.

- Les fichiers permanents ;
- Les fichiers mouvements ;
- Les fichiers de liaisons ;
- Les fichiers de manœuvres (fichiers de travail) ;
- Les fichiers tables.

- **Les fichiers permanents**

Ils contiennent des informations de base (fondamentales) pour l'application considérée. De plus, ces fichiers sont conservés de façon « permanent ».

Exemple : le fichier stock, le fichier client, le fichier fournisseur, le fichier des abonnés, etc.

Ils existent trois sous types de fichiers permanents :

- **Les fichiers des situations** : ils contiennent la « photographie » du contenu d'un fichier permanent à une date donnée ;
- **Les fichiers archives** : ce sont des fichiers situation que l'on conserve ;
- **Les fichiers historiques** : ils contiennent des « photographies » des fichiers permanents prises généralement à des intervalles de temps réguliers.

- **Les fichiers mouvements**

Ils contiennent les informations destinées à créer ou à mettre à jour des fichiers permanents.
Exemple : le fichier hebdomadaire des entrées et des sorties d'un stock.

- **Les fichiers de liaison**

Ils contiennent des informations qu'un programme communique à un autre programme. Ces programmes peuvent selon le cas :

- Appartenir à une même unité fonctionnelle ;
- L'un appartenir à une unité fonctionnelle d'une application, l'autre à une unité fonctionnelle de la même application ;
- L'un appartenir à une application, l'autre à une autre application.

Exemple : un fichier spool

- **Les fichiers de manœuvre (travail)**

Un fichier de manœuvre est un fichier qui est créé par un programme enfin de stocker provisoirement des informations qu'il utilise durant son exécution.
Avant la fin de son exécution, le programme considéré détruit généralement le ou les fichier(s) de manœuvres(s) utilisé(s).

- **Les fichiers tables**

Ce sont des fichiers qui contiennent des tableaux de valeurs, généralement utilisé par des programmes de calcul.
Exemple : fichier table des différents tarifs

b. Les taux d'activités

Les taux d'activités des fichiers sont :

- Le taux de consultation ;
- Le taux d'adjonction ;
- Le taux de suppression ;
- Le taux de mise à jour ;
- Le taux de croissance ;
- Le taux de rotation.

La connaissance de ces taux permet en particulier de mieux choisir les organisations de fichiers.

c. Le nombre de version de rétention

Pour des raisons de sécurité, il est souvent nécessaire de conserver plusieurs versions de la version originale d'un fichier. Ces versions sont appelées versions de rétention du fichier considéré.

Le contenu d'un fichier peut être endommagé par exemple par :

- Un incendie ;
- Une température excessive ;

- Une chute ;
- L'humidité ;
- La poussière ;
- Une tête de lecture mal réglée d'un périphérique de support magnétique, etc.

d. La date de création ; la fréquence de mise à jour, la période de rétention

Ces caractéristiques apparemment secondaires sont en réalité fondamentales.

- **La fréquence ou la périodicité de mise à jour**
  Elle intervient pour déterminer la fréquence de sauvegarde d'un fichier permanent. En effet plus la fréquence de mise à jour est grande plus le taux de mise à jour risque d'être important et donc plus la fréquence de la sauvegarde doit être grand.

- **La période de rétention d'un fichier**
  C'est la durée de vie du fichier.
  - Un fichier permanent a une durée de vie très longue ;
  - Un fichier mouvement a généralement une durée de vie égale au délai qui s'écoule entre deux sauvegardes des fichiers permanents qu'il sert à mettre à jour.

- **La date de création**
  La date de création d'un fichier doit être connu pour déterminer son âge.

- **La date de destruction**
  La date de destruction d'un fichier est obtenue grâce à sa date de création et à sa durée de vie.

e. Les niveaux d'élaboration d'un fichier

On dit qu'un fichier change de niveau d'élaboration chaque fois qu'il :

- Change de support ;
- Subit des modifications de contenu ;
- Est trié suivants de nouveaux critères.

Un niveau d'élaboration est un numéro ou code associé à un fichier afin de repérer les traitements qu'il a déjà subi.

f. Désignation (Modification des fichiers)

Il est vivement conseillé d'attribuer un nom (code) à chaque fichier de l'exploitation. Ainsi chaque fichier sera toujours désigné par le même nom durant toute l'étude de l'application, et dans tous les dossiers.

3. CARATERISTIQUES « PHYSIQUES » DES FICHIERS

a. le support

Les informations d'un fichier sont inscrites sur un support physique, communément appelé **le support**.

Nous distinguons deux types de support : **les supports non réutilisables et les supports réutilisables**.

Certains supports n'autorisent pas un programme, c'est à dire que les fichiers placés sur ces supports ne peuvent être utilisés qu'en mode lecture ou écriture, mais pas en mode lecture/écriture par un même programme.

b. La longueur, le blocage (groupage) et le format des données et des enregistrements des fichiers

- **La longueur**
  La longueur d'un enregistrement est égale à la somme des longueurs des informations élémentaires qu'il contient ;
- **Le blocage**
  A chaque opération de lecture ou d'écriture sur ce fichier, les informations sont transférées du support du fichier vers la mémoire centrale (ou de la mémoire centrale vers le support). Afin de diminuer le nombre de ces transferts, ces derniers doivent être réalisés non pas pour un enregistrement, mais pour un ensemble d'enregistrement consécutif du fichier considéré. L'ensemble des enregistrements transférés en une seule fois constitue **un bloc**.

Le nombre d'enregistrement contenu dans un bloc est appelé **le facteur de blocage** (ou **facteur du groupage**) du fichier.

Le facteur de blocage est une caractéristique d'un fichier.

Exemple :

Soit le fichier des 1200 employés de la Togolaise des eaux. Ce fichier contient un enregistrement par employé, soit 1200 enregistrements. Si les enregistrements sont bloqués avec un facteur de blocage de 10, chaque transfert entre la mémoire centrale et le support est réalisé pour 10 enregistrements consécutifs. Aussi pour lire tous les enregistrements de ce fichier, seulement 120 transferts sont effectués entre le support et la mémoire centrale.

4. PRINCIPAUX TRAITEMENT DES FICHIERS

a. Opérations élémentaires

La gestion des fichiers d'un point de vue algorithmique repose sur les opérations ou primitives élémentaires suivantes :

OPERATION EFFECTUEE	PRIMITIVE DE BASE
Ouverture	OUVRIR nom_fichier
Lecture	LIRE nom_enregistrement
Ecriture	Ecrire nom_enregistrement
Modification	REECRIRE nom_enregistrement
Suppression	SUPPRIMER nom_enregistrement
Positionnement	POSITION nom_enregistrement
Fermeture	FERMER nom_fichier

L'ensemble de ces opérations sera détaillé par la suite lors de l'étude plus approfondie de chaque organisation. Toutefois, voici quelques notions de base relatives à chacune de ces opérations élémentaires.

- **Ouverture**

Cette opération nécessaire avant l'utilisation de tout fichier permet d'indiquer le mode de traitement associé à ce fichier. Indiquée en début de programme, cette opération conditionne l'ensemble des opérations possibles dans la suite de l'algorithme. Le mode de traitement sera effectif jusqu'à ce que le fichier soit fermé. Des modes de traitements différents pouvant être utilisés pour un même fichier, il sera nécessaire en cas de mode opératoire de fermer puis d'ouvrir à nouveau le fichier.
Toute ouverture de fichier positionne un pointeur avant le premier enregistrement du fichier. L'emplacement du pointeur évolue au gré des différentes opérations.

- **Lecture**

Cette opération permet de lire un article du fichier en plaçant les données de l'articles dans les variables relatives à la structure du fichier (cf. définition la structure de fichier). Le transfert des données s'opère du fichier vers les variables en mémoire.

- **Ecriture**

Cette opération permet d'écrire un article dans le fichier en plaçant le contenu des variables relatives à ma structure du fichier dans l'article. Le transfert des données s'opère des variables vers le fichier.

- **Modification**

Cette opération permet de modifier l'article du fichier sur lequel est positionné le pointeur. Le transfert des données s'opère des variables vers les fichiers. Cette opération est obligatoirement précédée d'une opération de lecture.

- **Suppression**

Cette opération permet de supprimer l'article du fichier. La suppression peut être **logique** (l'article est marqué comme supprimé) ou **physique** (l'article est réellement supprimé du fichier).

- **Positionnement**

Cette opération permet de positionner explicitement le pointeur courant sur un article déterminé. Cette opération ne provoque pas de lecture de l'article.

- **Fermeture**

Cette opération permet de clore le fichier utilisé. Lors de la création ou de la mise à jour de fichier, elle permet de positionner une marque de fin de fichier. Cette opération a pour conséquence de rendre inaccessible le fichier concerné. Il est donc impossible d'effectuer une quelconque opération élémentaire sur ce fichier.

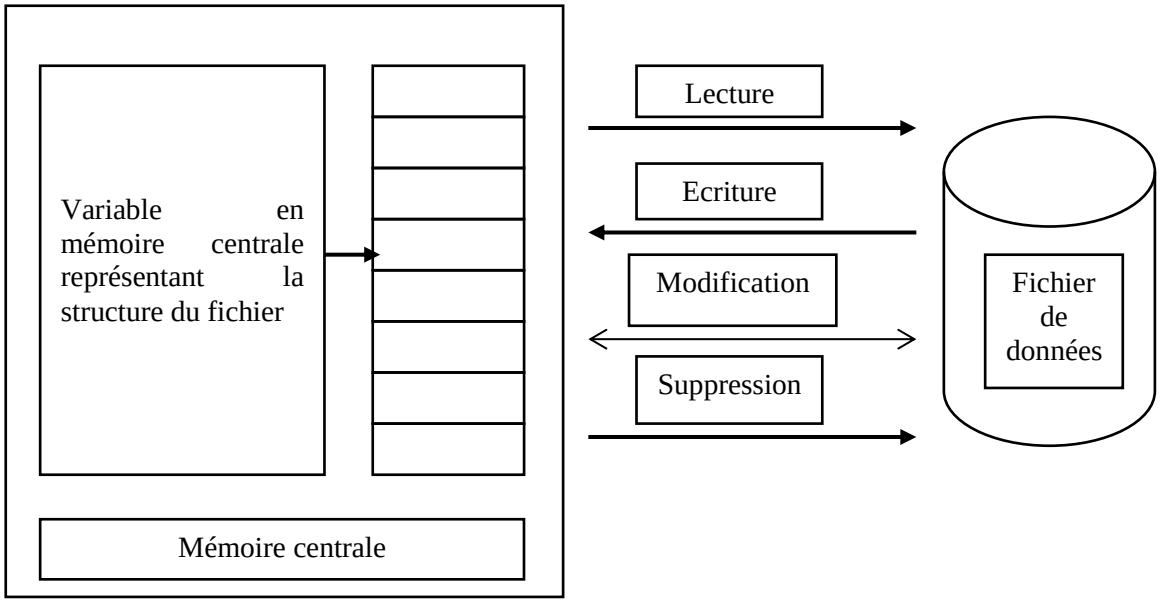


Figure 2: Schéma récapitulatif

b. Autres opérations élémentaires

- **Création**

  C'est l'opération qui consiste à mettre en place un fichier sur un support donné.
- **Tri**

  C'est l'opération consistant a ordonner un fichier en fonction d'un certain nombre de critères. Cette opération s'effectue souvent à l'aide d'une commande du système d'exploitation.
- **Mise à jour** : C'est l'opération qui consiste à actualiser un fichier. Elle est caractérisée par les trois opérations suivantes :
  - L'ajout d'un nouvel article ;
  - La modification d'un article existant ;
  - La suppression d'un article existant.

  En fonction de l'organisation choisie, ces opérations sont plus ou moins simples à réaliser.

- **Consultation**

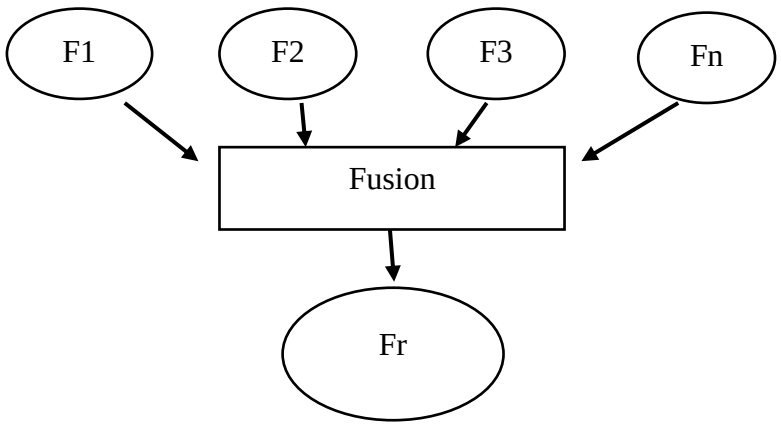
  C'est la demande d'informations sur un fichier.

L'ensemble de ces traitements s'appuie sur des opérations élémentaires.

c. Opérations concernant tous les articles d'un fichier

Certaines opérations concernent tous les articles, d'autre ne concernent que certains articles de fichier.

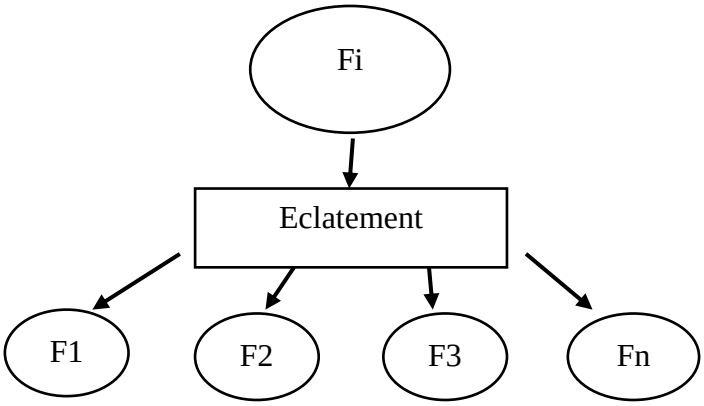
- **Création d'un fichier**
  Pour utiliser un fichier ; il faut que celui-ci existe, c'est à dire qu'il faut que les informations de ce fichier aient été stockées sur un support et soient utilisables.
- **Annulation (Destruction) d'un fichier**
  Cette opération est l'inverse de l'opération de création. Quand un fichier est annulé, il n'est plus possible de l'utiliser. On ne peut donc plus accéder à un quelconque de ces articles.
- **Tri d'un fichier (classement)**
  Cette opération consiste à classer tous les articles d'un fichier suivant les valeurs croissantes (ou décroissantes), d'une ou plusieurs informations qui figurent dans les articles de ce fichier. Les informations utilisées pour les tris sont appelées des **clés de tri** ou des **arguments de tri.**
  Si plusieurs clés sont utilisées pour tirer un fichier, on distingue :
  - **La clé majeure** : Clé grâce à laquelle le fichier est subdivisée en des ensembles d'articles ayant la même valeur de cette clé. Ces ensembles sont classés entre eux suivant les valeurs croissantes ou (décroissantes) de cette clé.
  - **La clé secondaire** : clé grâce à laquelle les articles de chacun des ensembles obtenus sont triés en sous ensemble de la même façon que précédemment.
  - **La clé tertiaire** : idem que précédemment, mais le tri est, cette fois ci, effectué à l'intérieur des sous-ensembles.
  - **La clé mineure** : dernière clé considérée pour le tri des articles du fichier considéré.
- **Révision et fusion de fichiers**
  Ces opérations permettent d'obtenir un fichier à partir de plusieurs autres fichiers.



On dit que l'on réalise une fusion quand on réunit plusieurs fichiers triés suivant le même critère.

- Eclatement d'un fichier

Cette opération permet d'obtenir plusieurs fichiers à partir du même fichier initial.



d. Opération concernant certains articles d'un fichier

- **Consultation d'un fichier**
  On dit que l'on consulte les articles d'un fichier (ou que l'on consulte un fichier) quand on lit certains articles de fichier.
- **Adjonction d'articles**
  Cette opération consiste à rajouter des articles à un fichier.
- **Suppression d'articles**
  Cette opération consiste à supprimer des articles d'un fichier.
  - Une suppression est **physique** quand les enregistrements sont réellement effacés du support du fichier
  - Une suppression est **logique** quand les enregistrements sont toujours présents sur le support après la suppression. Seule une variable booléenne est basculée afin de préciser qu'un enregistrement ait été supprimé.
- **Mise à jour des articles – Maintenance d'un fichier**
  La mise à jour au sens stricte consiste à modifier le contenu d'un ou de plusieurs articles d'un fichier.
  La mise à jour au sens large appelée maintenance d'un fichier revient à considérer à la fois les adjonctions d'articles, des suppressions d'articles et des modifications d'articles.

Chapitre 4 : LES PROCÉDURES ET LES FONCTIONS

I. LES PROCÉDURES

1. NOTION D'ACTION NOMMÉE OU DE PROCÉDURE

Il est assez désagréable d'avoir à lire un algorithme écrit sur plusieurs pages. Or peu de problèmes peuvent être résolus en moins d'une page d'algorithme. Pour des raisons de lisibilité, l'on est donc amené à découper un algorithme en plusieurs parties :

- Une partie principale indiquant la logique générale de l'algorithme et
- Plusieurs sous parties éventuellement décomposées en autant de sous parties que nécessaires.

Tous ces sous partis portent le nom **d'actions nommées ou de procédures**.

Exercice d'application : *voir cahier d'exercices*

2. NOTIONS RELATIVES AUX ARGUMENTS

Lorsqu'une action nommée apparaît dans la description d'un algorithme, on dit qu'elle est appelée. Une action nommée utilise éventuellement des arguments ou paramètres. Ces arguments ou paramètres appelés également variables publiques de l'action nommée, constitue son interface. Les autres variables propres à l'action nommée, ne sont pas « visibles » par les actions appelantes et sont appelées variables locales ou privées.
Décrire une action nommée, c'est spécifier les données et résultats de l'action à l'aide des arguments formels.
Appeler une action nommée se fait en fournissant la liste des objets appartenant à l'action appelante. Cette liste constitue les arguments effectifs qui se substitueront aux arguments formels pour l'exécution de l'action appelée.

Déclaration d'une procédure ou action nommée

PROCEDURE LibelleAction (liste des **arguments formels**)
VAR (liste des **variables locales ou privées**)
DEBUT
…
FIN

Appel d'une procédure dans l'action appelante

LibelleAction (liste des **paramètres effectifs**).

Il est important de noter que les paramètres effectifs doivent avoir exactement le même type et de préférence la même signification que les paramètres formels utilisés dans l'en-tête de l'action nommée. Par contre, ils n'ont pas nécessairement le même nom.

Les paramètres effectifs sont transmis à l'action nommée au moment où elle est appelée ou « activée ». L'action nommée va éventuellement effectuer des calculs sur ces arguments. Les possibilités d'opérer sur les arguments effectifs dépendent en fait du mode de passage des arguments. Une des difficultés de la notion d'action nommée paramétrée provient du fait qu'il y

a plusieurs mécanismes de passage des arguments entre une action appelante et une action appelée et que tous les langages de programmation n'offrent pas tous, les mêmes possibilités. De plus, un mode de passage est souvent sous-entendu : il faut donc garder à l'esprit son mode de fonctionnement pour ne pas faire d'erreurs.

SYNTAXES RELATIVES AUX PROCEDURES

Déclaration de la procédure

PROCEDURE « nom de la procedure » (« argument » ; « arguments » *)

{« déclarations »} *

DEBUT

{« instructions »} *

FIN

Argument ::= <type_passage> ,<argument> * : <type>

Type_passage ::= e | s | es

Argument ::= <identificateur de variable>

Appel d'une procédure dans une action appelante

« nom de la procédure » (liste des paramètres effectifs)

2.3 PASSAGE DES PARAMÈTRES

Le type de passage permet d'indiquer la façon dont sont passés les arguments.

- **Argument en entrée (e)** : un argument passé en « entrée » signifie que l'on ne souhaite pas modifier le contenu initial de la variable transmise. Le passage par adresse est également possible.
- **Argument en sortie (s)** : un argument passé en « sortie » signifie que l'on attend de la procédure appelée qu'elle initialise la variable transmise. Le mode de passage retenu est obligatoirement le passage par adresse.
- **Argument en entrée-sortie (es)** : un argument passé en « sortie » signifie que l'on attend de la procédure appelée qu'elle utilise puis qu'elle modifie la variable transmise. Le mode de passage retenu est obligatoirement le passage par adresse.

II. LA NOTION DE FONCTION

DÉFINITION

La fonction est une action nommée similaire à la procédure mais qui retourne une valeur d'un type donné. La fonction dans une action appelante est toujours utilisée dans une expression (arithmétique ou logique). On distingue :

- Les fonctions utilisateurs ;
- Les fonctions standards ;
- Les Fonctions Utilisateurs.

Ce sont les fonctions définies par l'utilisateur. Elles obéissent à la syntaxe suivante :

Syntaxe

FONCTION « nom de la fonction » (« arguments » *) : type

{« déclarations »} *

DEBUT
{« instructions »} *
délivre (expression résultat) | **valret** :=expression résultat

FIN

Arguments ::= <argument> ,<argument> * : <type>
Argument ::= <identificateur de variable>

Les arguments formels sont implicitement déclarés en entrée car une fonction ne doit pas à priori modifier un paramètre formel.

Pour qu'une fonction puisse retourner la valeur calculée au programme appelant, il est nécessaire d'avoir dans le corps de la fonction au moins une instruction d'affectation indiquant la valeur retournée. Cette instruction est « délivre (expression résultat) » ou « valret : = expression résultat ».

Appel d'une fonction dans une action appelante

<nom de la fonction> (liste des paramètres effectifs)

1. LES FONCTIONS STANDARDS

Ce sont les fonctions de base offertes par le langage utilisé (langage algorithmique ou de programmation). Leur nombre et leur spécification sont fonction du langage utilisé. On peut considérer une fonction comme un nouvel opérateur du langage qui permet de calculer un résultat. De ce fait elles sont directement utilisables sans avoir besoin de les déclarer.

Syntaxe : Appel d'une fonction standard

Une fonction standard est toujours utilisée dans une expression conformément à la syntaxe suivante :

<nom de la fonction> (<paramètre effectif> , < paramètre effectif > *))

2. FONCTIONS ET PROCÉDURES RÉCURSIVES

Une fonction ou une procédure est récursive si elle s'appelle elle-même directement ou indirectement.

Exemple : Ecrivons la fonction factorielle de façon récursive

```
Fonction factorielle (Nombre : entier) : entier

Début
    Si Nombre = 0 alors
        Valret := 1
    Sinon
        Valret :=Nombre * factorielle (Nombre – 1)
Fin
```

III. LA PROGRAMMATION PAR MODULES

1. UTILISATION DES ACTIONS NOMMÉES

Il est important de bien comprendre que l'usage des actions nommées représente la base de la programmation structurée, qui consiste d'abord à décrire ensuite les actions nommées par leur spécification, puis à en décrire ensuite leur comportement.

Intuitivement, on voit bien que cette façon de faire permet d'une part de maîtriser la résolution d'un problème complexe en l'assimilant à la résolution d'une suite de problèmes plus simples, d'autre part d'utiliser les solutions apportées à certains problèmes pour en résoudre d'autres.

Ce sont là des facteurs fondamentaux dans l'efficacité de l'activité de programmation.

On peut en fait voir une action nommée de deux points diamétralement opposés, mais complémentaires :

- **De l'extérieur**
  On utilise l'action comme une boîte noire, dont on connaît le rôle, grâce à ses spécifications, et les objets qu'elle peut manipuler, mais dont on ne connaît pas le principe de fonctionnement.
- **De l'intérieur**
  On en connaît le rôle, les types d'objets qu'elle peut manipuler et le principe de fonctionnement mais on ne sait pas où elle sera utilisée. Lors de la construction de programmes, cette technique permet de tester d'abord les actions nommées, avant de les insérer dans le programme qui les utilise.

En résumé, les actions nommées permettent :

- De partager la complexité en abordant un problème d'un niveau global vers un niveau détaillé.

- De réutiliser n fois un même module, plutôt que d'écrire n fois les mêmes instructions. La réécriture d'instructions identiques doit d'ailleurs interpeller systématiquement le concepteur quant à la mise en place d'une action nommée.
- De maintenir plus facilement un programme, puisque la modularité permet de repérer plus facilement tout dysfonctionnement.
- De faire appel à des bibliothèques de modules existants dont on ne connaît que la partie « spécifications ».
- De répartir la charge d'un projet de développement conséquent entre plusieurs personnes.

La décomposition d'un algorithme en actions nommées est une tâche difficile, qui repose sur une capacité à « aller à l'essentiel ».

2. QUELQUES CONSEILS POUR FACILITER LA DÉCOMPOSITION

Une action doit toujours jouer un rôle précis et facile à définir.

L'interface entre les actions doit être la plus réduite possible : en général, une action ne doit comporter que quelques arguments.

Il faut donc décomposer suffisamment pour réduire la complexité du problème, mais pas trop pour ne pas augmenter la complexité de la solution !